

APIOBike

APIOBike es un servicio de bicicletas recién lanzado en la Ciudad APIO. En la ciudad se han construido varias estaciones, lo que le permite a los usuarios tomar prestadas o regresar bicicletas en cualquier estación. Debido a su conveniencia y su bajo costo, se ha vuelto el medio de transporte más importante en la Ciudad APIO. Sin embargo, APIOBike se enfrenta a un problema común de todos los servicios públicos de bicicletas: las tasas de préstamo y devolución en distintas estaciones están desequilibradas. Esto provoca que en algunas estaciones haya muy pocas bicicletas para la demanda local, mientras que en otras estaciones hay más bicicletas de las que necesita la gente de la zona.

Para remediar este problema, la Ciudad APIO planea enviar un camión de carga cada noche para redistribuir las bicicletas y asegurar que cada estación tenga una cantidad apropiada de bicicletas. La Ciudad APIO tiene un total de N estaciones numeradas $0, 1, \dots, N - 1$. APIOBike se ha dado cuenta que cada noche, la cantidad de bicicletas en la estación i siempre es un valor fijo $A[i]$ y que nadie toma prestada o regresa una bicicleta durante la noche. La compañía desea tener $B[i]$ bicicletas en la estación i cada mañana.

Hay $N - 1$ caminos entre las N estaciones por los que puede pasar el camión. Cada camino conecta dos estaciones distintas y mide una unidad de largo. La red de estaciones es tal que el camión puede viajar entre cualquier par de estaciones a través de los caminos. Cada noche, APIOBike debe mandar un camión para asegurar que cada estación i tenga exactamente $B[i]$ bicicletas. El camión puede empezar en cualquier estación y puede terminar su ruta en cualquier estación.

El camión está vacío al empezar la redistribución. Siempre que el camión está en una estación, el conductor puede subir la cantidad de bicicletas que quiera de la estación al camión. De forma parecida, también puede bajar la cantidad de bicicletas que él quiera del camión y dejarlas en la estación. Tanto el camión como las estaciones tienen capacidad infinita, pero la cantidad de bicicletas en cualquier lugar no puede ser menor a cero en ningún momento. La compañía quiere determinar la mínima distancia de viaje con la que puede completar la redistribución junto con una estrategia correspondiente.

Detalles de implementación

Deberás implementar la siguiente función.

```
std::pair<std::vector<int>, std::vector<long long>>
find_rebalancing_strategy(int N,
                           std::vector<int> A,
                           std::vector<int> B,
                           std::vector<int> U,
                           std::vector<int> V)
```

- A : un arreglo de longitud N , en el que $A[i]$ es la cantidad de bicicletas en la estación i cada noche.
- B : un arreglo de longitud N , en el que $B[i]$ es la cantidad de bicicletas que se quiere tener en la estación i cada mañana.
- U, V : arreglos de longitud $N - 1$ que representan la red de caminos. Para cada $0 \leq i < N - 1$, el i -ésimo camino conecta la estación $U[i]$ y la estación $V[i]$.
- Esta función se llama **a lo más 150 000 veces** por cada caso de prueba.

La función deberá devolver un par de arreglos (X, Y) de longitud $k + 1$ que representan una estrategia de redistribución:

- X : la secuencia de índices de las estaciones que se visitan en orden cronológico.
- Y : la entrega neta de bicicletas en cada estación visitada. Para cada $0 \leq j \leq k$:
 - Si $Y[j] \geq 0$, el conductor baja $Y[j]$ bicicletas en la estación $X[j]$ del camión, incrementando en $Y[j]$ la cantidad de bicicletas en la estación.
 - Si $Y[j] < 0$, el conductor sube $-Y[j]$ bicicletas de la estación $X[j]$ al camión, disminuyendo en $-Y[j]$ la cantidad de bicicletas de la estación.

Una estrategia **válida** de redistribución (X, Y) con distancia de viaje k debe cumplir las siguientes condiciones:

- Para cada $0 \leq j \leq k$, $0 \leq X[j] < N$.
- Para cada $0 \leq j < k$, stations $X[j]$ and $X[j + 1]$ are directly connected by a road.
- Para cada $0 \leq j \leq k$, $\sum_{t=0}^j Y[t] \leq 0$.
- Para cada estación $0 \leq i < N$ y cada paso $0 \leq j \leq k$, sea $S_{i,j}$ la suma de $Y[t]$ sobre todos los pasos t tales que $0 \leq t \leq j$ and $X[t] = i$.
 - La cantidad de bicicletas en la estación i nunca es menor a cero: $A[i] + S_{i,j} \geq 0$.
 - En el momento en el que la estrategia termina, cada estación i debe contener exactamente $B[i]$ bicicletas: $A[i] + S_{i,k} = B[i]$.

Consideraciones

- $2 \leq N \leq 300\,000$

- La suma de N sobre todas las llamadas a `find_rebalancing_strategy` no excede 300 000 en cada caso de prueba.
- $0 \leq U[i], V[i] < N$ y $U[i] \neq V[i]$ para cada i tal que $0 \leq i < N$.
- $0 \leq A[i], B[i] \leq 10^9$ para cada i tal que $0 \leq i < N$.
- Es posible viajar entre cada par de estaciones.
- $\sum_{i=0}^{N-1} A[i] = \sum_{i=0}^{N-1} B[i]$.
- Existe al menos un i tal que $0 \leq i < N$ and $A[i] \neq B[i]$.

Subtareas y puntuación

Sea T la cantidad de llamadas a `find_rebalancing_strategy`, y $\sum N$ la suma de todos los N sobre todas las llamadas a `find_rebalancing_strategy`.

Subtarea	Puntuación	Restricciones adicionales
1	4	Las estaciones y los caminos satisfacen la propiedad P (ver abajo). Existe exactamente un i tal que $0 \leq i < N$ y $A[i] > 0$.
2	11	$T \leq 10$. $N \leq 7$. Las estaciones y los caminos satisfacen la propiedad P.
3	16	Las estaciones y los caminos satisfacen la propiedad P.
4	9	Existe exactamente una i tal que $0 \leq i < N$ y $A[i] > 0$.
5	24	$\sum N \leq 500$.
6	15	$\sum N \leq 5000$.
7	21	Sin restricciones adicionales.

Propiedad P: Para cada $0 \leq i < N - 1$, $U[i] = i$ y $V[i] = i + 1$. Para cada $0 \leq i < N$, $A[i] \neq B[i]$.

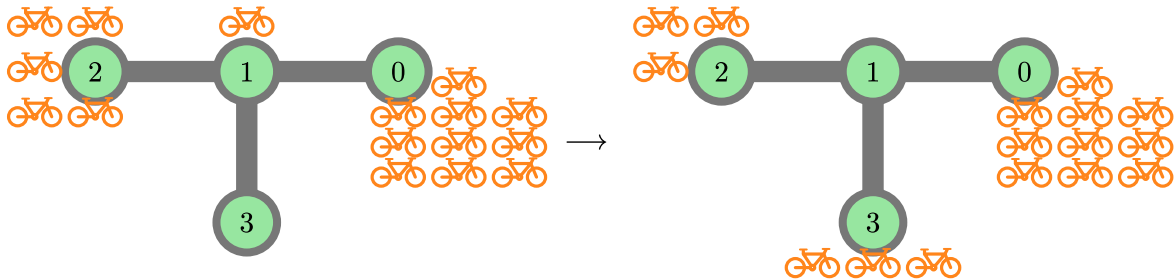
Si la longitud de los arreglos retornados X and Y es exactamente $k^* + 1$, donde k^* es la mínima distancia de viaje posible, pero (X, Y) no es una estrategia válida, se te dará el 50% de la puntuación.

Ejemplo

Ejemplo 1

Considera la siguiente llamada:

```
find_rebalancing_strategy(4,
                        [10, 1, 5, 0],
                        [10, 0, 3, 3],
                        [0, 1, 1],
                        [1, 2, 3])
```



Hay $N = 4$ estaciones en la ciudad APIO. Al inicio, las estaciones tienen $A = [10, 1, 5, 0]$ bicicletas y el objetivo es tener $B = [10, 0, 3, 3]$ bicicletas en cada estación.

Supón que el camión empieza en la estación 2 y lleva a cabo los siguientes pasos:

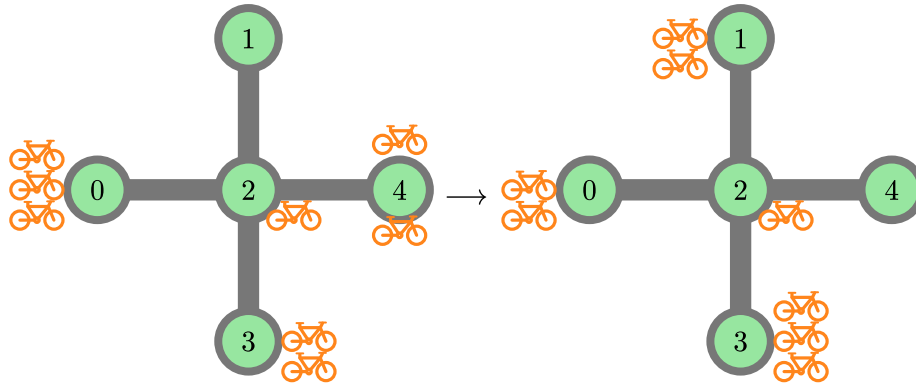
- En la estación 2, sube 2 bicicletas al camión. La estación 2 ahora tiene 3 bicicletas y el camión tiene 2 bicicletas.
- Se mueve a la estación 1 y sube 1 bicicleta al camión. La estación 1 ahora tiene 0 bicicletas y el camión tiene 3 bicicletas.
- Se mueve a la estación 3 y baja 3 bicicletas. La estación 3 ahora tiene 3 bicicletas y el camión tiene 0 bicicletas.

La distancia de viaje total es 2. Los arreglos correspondientes a devolver por la estrategia son $X = [2, 1, 3]$ y $Y = [-2, -1, 3]$. Se puede demostrar que esta estrategia logra la distancia mínima. Por lo tanto, la función debe retornar $([2, 1, 3], [-2, -1, 3])$.

Ejemplo 2

Considera la siguiente llamada:

```
find_rebalancing_strategy(5,
                        [3, 0, 1, 2, 2],
                        [2, 2, 1, 3, 0],
                        [2, 2, 2, 2],
                        [0, 4, 3, 1])
```



Hay $N = 5$ estaciones en la Ciudad APIO. Inicialmente, las estaciones tienen $A = [3, 0, 1, 2, 2]$ bicicletas y el objetivo es tener $B = [2, 2, 1, 3, 0]$ bicicletas en cada estación.

Supón que el camión empieza en la estación 0 y lleva a cabo los siguientes pasos:

- En la estación 0, sube 1 bicicleta.
- Se mueve a la estación 2 y sube 1 bicicleta.
- Se mueve a la estación 1 y baja 2 bicicletas.
- Se mueve a la estación 2.
- Se mueve a la estación 4 y sube 2 bicicletas.
- Se mueve a la estación 2 y baja 1 bicicleta.
- Se mueve a la estación 3 y baja 1 bicicleta.

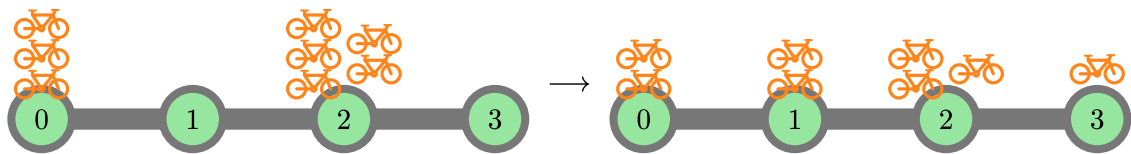
La distancia de viaje total es 6. Los arreglos correspondientes a devolver por la estrategia son $X = [0, 2, 1, 2, 4, 2, 3]$ y $Y = [-1, -1, 2, 0, -2, 1, 1]$. Se puede demostrar que esta estrategia logra la distancia mínima. Por lo tanto, la función debe retornar $([0, 2, 1, 2, 4, 2, 3], [-1, -1, 2, 0, -2, 1, 1])$.

Otras estrategias válidas con distancia 6, como $X = [0, 2, 3, 2, 4, 2, 1]$ y $Y = [-1, 0, 1, 0, -2, 0, 2]$, también se consideran correctas. Si devuelves arreglos X, Y con longitud $7 = 6 + 1$ pero sin una estrategia válida, como $X = Y = [0, 0, 0, 0, 0, 0, 0]$, se te dará el 50% de la puntuación.

Ejemplo 3

Considera la siguiente llamada:

```
find_rebalancing_strategy(4,
                        [3, 0, 5, 0],
                        [2, 2, 3, 1],
                        [0, 1, 2],
                        [1, 2, 3])
```



Una solución óptima es $X = [2, 1, 0, 1, 2, 3]$ y $Y = [-1, 1, -1, 1, -1, 1]$. La función debe retornar $([2, 1, 0, 1, 2, 3], [-1, 1, -1, 1, -1, 1])$. Otras estrategias válidas, como $X = [2, 1, 0, 1, 2, 3]$ y $Y = [-2, 2, -1, 0, 0, 1]$, también se consideran correctas.

Este ejemplo satisface las restricciones de las subtarear 2 y 3.

Evaluador de ejemplo

La primera línea de la entrada debe contener un solo entero T , la cantidad de escenarios. Le debe seguir una descripción de T escenarios, cada uno en el formato que se especifica a continuación.

Formato de entrada:

```
N
A[0] A[1] ... A[N-1]
B[0] B[1] ... B[N-1]
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
```

Formato de salida:

```
k
X[0] X[1] ... X[k]
Y[0] Y[1] ... Y[k]
```